

Imp # Recursion

Recursion is a process in which a function calls itself directly or indirectly to solve a problem. A recursive function must have a base condition to stop the recursion.

Recursive Process = Base Case + Recursive Case

Basic Structure in C

```
return-type function()  
{  
    if (condition)           } Base Case  
        return;  
  
    function(); /// } Recursive Case  
}
```

Program - Factorial

```
#include <stdio.h>
```

```
int factorial (int n)
```

```
{  
    if (n == 0)  
        return 1;
```

```
    return n * factorial (n - 1);  
}
```

```
int main()  
{  
    int n=5;  
    printf("Factorial is %d", factorial(n));  
    return 0;  
}
```

Output - Factorial

Factorial is 120

Program - Sum of n Numbers

```
#include <stdio.h>  
  
int sum(int n)  
{  
    if (n == 0)  
        return 0;  
    return n + sum(n-1);  
}  
  
int main()  
{  
    printf("Sum is %d", sum(5)); // Sum is 15  
    return 0;  
}
```

Simulating Recursion

Simulating recursion means performing the same work as a recursive function without actually using recursive function calls.

Recursion internally uses a stack to keep track of function calls. We could manually use stack to simulate this process.

Backtracking

Backtracking is a problem solving technique in which we build a solution step by step and go back when the current choice does not lead to a valid solution.

Backtrack

Choose → Explore → If wrong, undo → Try Another Choice

Example: Finding Path in a Maze

- Moves to a possible position.
- Continue moving forward.
- If we reach a dead end, go back to the previous position.
- Try another path.

Backtracking = Recursion + Undo when the choice is wrong

Basic Structure of Backtracking in C:

```
void solve()  
{  
    if(solution_found)  
        return;  
  
    for (each_possible_choice)  
    {  
        if (choice_is_valid)  
        {  
            make_choice();  
            solve();  
            undo_choice();  
        }  
    }  
}
```

Applications of Backtracking:

- N-Queens Problem
- Maze Solving
- Sudoku
- Graph Coloring
- Generating Permutations.

Tail Recursion

Tail recursion is a type of recursion in which the recursive call is the last operation performed by the function. Simply it means after the recursive call nothing remains to be done.

Program:

```
#include <stdio.h>
```

```
void print_number(int n)
```

```
{  
    if (n == 0)  
        return;
```

```
    printf("%d ", n);
```

```
    printf(n +
```

```
    print_number(n - 1); // Last Operation  
}
```

```
int main()  
{
```

```
    print_number(5);  
    return 0;
```

```
}
```

Output:

5 4 3 2 1

Removal of Recursion

Removal of recursion means converting a recursive function into an iterative function using loops such as for, while or do while. This is called recursion elimination.

Example Program for Factorial:

```
int factorial (int n)
{
    if (n == 0)
        return 1;
    return n * factorial (n-1);
}
```

Recursive Method:

```
int factorial (int n)
{
    int fact = 1;
    for (int i = 1; i <= n; i++)
        fact = fact * i;
    return fact;
}
```

Imp ## Tower of Hanoi

Tower of Hanoi is a classic problem that can be solved using recursion.

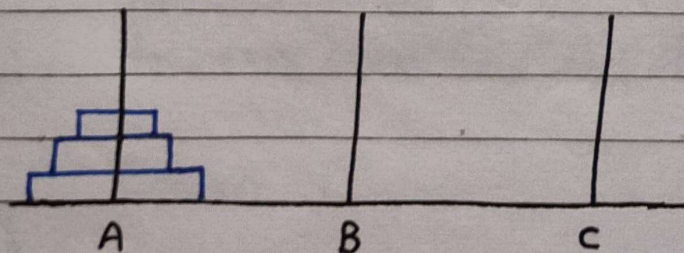
It consists of:

- 3 rods - Source, Auxiliary & Destination.
(A) (B) (C)
- n -disks of different sizes.
- All disks are initially placed on the Source rod, with the largest at the bottom.

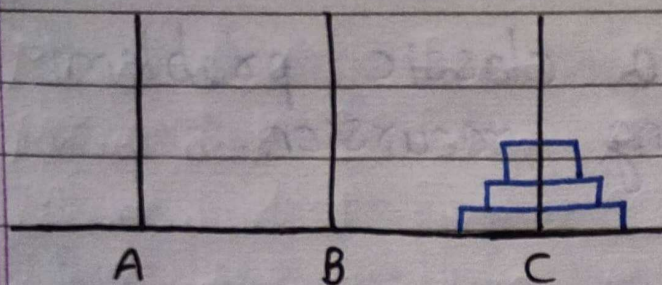
Rules:

- Only one disk can be moved at a time.
- Only the top disk of a rod can be moved.
- A larger disk cannot be placed on a smaller disk.

Initial:



Goal:



Move Count:

The minimum number of moves needed is $2^n - 1$, where n is the number of disks.

For example; 3 disks take 7 moves,
4 disks take 15 moves,
5 disks take 31 moves, etc.

Recursive Logic:

• To move n disks from Source [A] to Destination [C],

1. Move $n-1$ disks from Source to Auxiliary.
2. Move the largest disk from Source to Destination.
3. Move $n-1$ disks from Auxiliary to Destination.

Time Complexity:

$$O(2^n).$$

Space Complexity:

$$O(n).$$

Program for Tower of Hanoi:

```
#include <stdio.h>
```

```
void towerofhanoi (int n, char source, char  
auxiliary, char destination)  
{
```

```
    if (n == 1)  
    {
```

```
        printf("Move disk 1 from %c to  
               %c \n", source, destination);
```

```
        return;
```

```
    towerofhanoi (n-1, source, destination, auxiliary);
```

```
    printf("Move disk %d from %c to %c \n",  
           n, source, destination);
```

```
    towerofhanoi (n-1, auxiliary, source, destination);  
}
```

```
int main()  
{
```

```
    int n = 3;
```

```
    towerofhanoi(n, 'A', 'B', 'C');
```

```
    return 0;
```

```
}
```

Output:

Move disk 1 from A to C

Move disk 2 from A to B

Move disk 1 from C to B

Move disk 3 from A to C

Move disk 1 from B to A

Move disk 2 from B to C

Move disk 1 from A to C

3 DISKS

